IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 6 February 2026, accepted 23 February 2026, date of publication 26 February 2026, date of current version 5 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3668729

RESEARCH ARTICLE

AI-Enhanced Metaheuristics Control for Project Scheduling Using Neural Networks

NIKOLAOS T. MITSOU AND GEORGIOS K. KOULINAS
Democritus University of Thrace, 67100 Xanthi, Greece
Corresponding author: Nikolaos T. Mitsou (nmitsou@pme.duth.gr)

ABSTRACT Project management requires systematically planning and allocating time, resources, and people to complete complex projects. A well-known problem in project management is the Resource-Constrained Project Scheduling Problem (RCPSP), which seeks to minimize total project duration (makespan) while respecting limited resource availability and activity durations. The RCPSP is categorized as an NP-hard problem, so no polynomial-time solution is known. Artificial intelligence is transforming project management by automating and optimizing planning, execution, and control to boost project success. In recent years, extensive research has been conducted on AI techniques aimed at improving project task scheduling processes. In task scheduling, researchers employ nature-inspired metaheuristics, hyper-heuristics, reinforcement learning, and hybrid techniques (e.g., GA, Bee algorithms, Simulated Annealing, MDP/DRL approaches). The main contribution of the present study is the development of an artificial intelligence model designed to select the most appropriate algorithm for project scheduling problems with specific characteristics. This model consists of a neural network that takes as input the features of the project and outputs a recommendation for the most suitable algorithm for scheduling the project under consideration. The algorithms considered by the model include a Genetic Algorithm, a Simulated Annealing algorithm, a Bee Colony Optimization algorithm, as well as a priority rule based heuristic algorithm. Each of these methods offers distinct advantages depending on the nature of the project, such as its size, complexity, constraints, and resource limitations. The AI model is trained to recognize which algorithm is likely to yield the most effective scheduling results under different scenarios.

INDEX TERMS Neural networks, algorithmic selection, project scheduling, metaheuristics, optimization.

I. INTRODUCTION

The Resource-Constrained Project Scheduling Problem (RCPSP) was first introduced by Pritsker in 1969 and has since been extensively studied by numerous researchers. As a well-known NP-hard problem [1] it belongs to the class of non-deterministic polynomial-time problems, making it inherently difficult to solve optimally. Due to this complexity, a wide range of metaheuristic algorithms have been developed to approximate solutions to the problem, particularly in projects with many activities [2], [3]. Metaheuristic algorithms designed for solving the RCPSP adopt various operational strategies and can be categorized into three main groups: local search-based metaheuristics (e.g., Simulated Annealing, Tabu Search), population-based metaheuristics (e.g., Bee Colony Optimization, Genetic Algorithm, Particle Swarm Optimization), and learning-based metaheuristics, which incorporate models such as neural networks [4]. These techniques aim to deliver optimal or near-optimal solutions within reasonable computational times, addressing the challenges posed by the problem's combinatorial nature.

Since the introduction of the problem in 1969 [5], various methods have been proposed to solve it. These include heuristic methods, which were primarily developed up until the 1990s, and metaheuristic approaches, which have seen significant advancement over the past two decades. Heuristic methods rely on schedule generation schemes, either serial or parallel, combined with priority rules to construct schedules, with an emphasis on computational speed [6]. Meta-heuristics follow rules to thoroughly explore the most

The associate editor coordinating the review of this manuscript and approving it for publication was Valerio Freschi.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

"promising areas" of the solution space in order to find high-quality solutions [7]. The metaheuristic algorithms developed for solving RCPSP follow different operational strategies and can be categorized into local search metaheuristics (Simulated Annealing, Tabu Search, etc.), population-based metaheuristics (Bee Colony Optimization, Genetic Algorithm, Particle Swarm Optimization), and the class of metaheuristics - learning metaheuristics that are neural network models [4]. Although metaheuristics have improved solution quality, they frequently require problem-specific design and tuning. Hyper-heuristics address this gap by operating at a higher level to select, combine, or generate lower-level heuristics, improving adaptability across diverse RCPSP instances [8], [9]. These hyper-heuristic approaches have been implemented in many meta-heuristic algorithms over time. For instance, Anagnostopoulos and Koulinas, Dowsland and Burke proposed a Simulated Annealing hyper-heuristic Algorithm [10], [11]. Additional applications of hyper-heuristic algorithms have been employed in many algorithms like PSO [3], Tabu Search [12] and Genetic Algorithm [13], [14], [15], [16].

The algorithms employed in this study for treating the RCPSP include a Genetic Algorithm, the Simulated Annealing algorithm, a Bee Colony Optimization metaheuristic, and a Priority Rule based Heuristic Algorithm, that constructs solutions through the random application of priority rules.

The Genetic Algorithm was firstly introduced in the context of RCPSP by [17]. It is a population-based method that leverages evolutionary theory, aiming to converge toward an optimal or near-optimal solution through successive generations [18]. As one of the most prominent solution methods for the RCPSP, Genetic Algorithm has undergone continuous improvements. Notably, [19] introduced two scheduling tactics incorporating resource buffers, while [20] enhanced the Genetic Algorithm by integrating alternative subgraphs and applying a satisfiability.

The Simulated Annealing (SA) algorithm was initially developed by [21], and later adapted for the RCPSP by [22]. SA belongs to the category of local search algorithms and is inspired by physical processes, particularly the annealing process in metallurgy [23].

The Bee Colony Optimization metaheuristic was originally proposed by [24]. Since then, several researchers, including Shi et al. ([25]), as well as Akbari and Zaighami [26], have modified this algorithm—also known as Artificial Bee Colony (ABC)—for application to the RCPSP. The Bee Colony Optimization algorithm belongs to the class of population-based algorithms and more specifically to the family of swarm intelligence optimization techniques [27].

Learning-based metaheuristics have also emerged as a powerful strategy for addressing optimization problems, utilizing artificial intelligence and machine learning methods [28], [29]. These approaches typically consist of hybrid algorithms that combine metaheuristic techniques with machine learning and data mining methods to enhance the capability and efficiency of optimization algorithms [28].

For the RCPSP, [30] proposed an artificial neural network (ANN) model that generates project schedules by selecting among three priority rules: Latest Finish Time (LFT), Minimum Slack Time (MST), and Maximum Remaining Path Length (MRPL). Similarly, [31] introduced four ANN models, each with a different structure, to select activity priority rules for the projects under consideration. Along the same lines, [32] proposed a deep reinforcement learning (DRL) model—specifically, a Graph Neural Network (GNN)—which selects priority rules. Golab and Gooya also developed ANN-based models, including a Multi-Layer Feedforward Neural Network (MLFNN) and a Convolutional Neural Network (CNN), that select activity priority rules based on project features at each scheduling step [33], [34]. Finally, [35] proposed a model based on reinforcement learning and GNNs designed to map scheduling policies under conditions of resource availability disruptions. The above studies highlight the increasing use of neural networks in addressing the RCPSP, particularly in identifying appropriate priority rules. Since priority rules are fundamental to project scheduling, the academic community continues to publish research proposing new approaches and conducting further investigations in this direction [36], [37], [38], [39].

Metaheuristic algorithms are powerful optimization tools, designed to be problem-independent and therefore applicable to a wide range of optimization problems. This broad applicability has led researchers to develop algorithm selection methods. The aim of these methods is to identify the most suitable algorithm for solving a given optimization problem, rather than designing new algorithms for each specific case [40]. The algorithm selection problem was first introduced by [41], who defined a problem space and an algorithm space, proposing that each problem-algorithm pair can be mapped based on performance. This mapping enables the selection of the most effective algorithm for a specific problem. Since then, algorithm selection models have been developed for a variety of optimization problems, including bin packing, the competitive travelling salesman problem, exam timetabling, and flow shop scheduling [40].

For the RCPSP specifically, [42] proposed an automatic selection approach that chooses between two algorithms. This method effectively forms a hyper-heuristic, which performs better than either of the two algorithms when used individually.

II. PROBLEM FORMULATION

The Resource-Constrained Project Scheduling Problem (RCPSP) is considered one of the most challenging problems in project management, where finding or approximating the optimal solution holds significant value. The objective of the RCPSP is to generate a project schedule—that is, to determine the start and finish times of each activity—with the goal of minimizing the total project duration while satisfying

resource constraints and precedence relations among activities. The RCPSP problem, when formulated as a project makespan minimization problem under resource constraints, belongs to the class of NP-Hard problems [43]. The formal proof of this classification was presented earlier by Blazewicz [1].

The mathematical formulation of the problem, as presented by Kolisch, is as follows:

$$minf_n \tag{1}$$

$$f_i \leq f_i - d_j \, for \, all \, precedence \, relations \tag{2}$$

$$f_1 = 0, d_1 = 0, d_n = 0 \tag{3}$$

$$\sum_{i \in P_t} u_{irt} \leq U_r \, for \, each \, t \; = 1, \ldots, f_n \, and \, r \in R \tag{4}$$

- The objective of the problem is to find a solution that minimizes the total project duration (makespan) – f_n function.
- The second constraint ensures that precedence relations among project activities are respected during scheduling.
- The third constraint sets the project start time at time zero.
- The fourth constraint defines that the availability of resource U remains constant throughout the project's duration and cannot be exceeded. Specifically, the total demand u_{irt} from all activities P_t at any given time t must not exceed the resource availability U_r.

III. THE CONCEPT OF NEURAL NETWORK FOR EVALUATING AND SELECTING METAHEURISTICS

Neural networks are computational models inspired by the structure and function of the human brain, consisting of interconnected neurons organized in layers. This architecture enables neural networks to learn patterns and discover relationships within large datasets, in a manner similar to the human brain. This ability is utilized in the present study to evaluate the results of metaheuristic algorithms and to recommend the most suitable algorithm based on the characteristics of the project under consideration.

While previous approaches have been focused on the selection mechanism between two algorithms or the development of entirely new novel algorithms, the method proposed in this work extends the idea by employing an Artificial Neural Network (ANN) to choose among four distinct metaheuristic algorithms. Unlike the earlier studies the algorithms considered here differ in their operational strategies, thereby broadening the search landscape. This not only increases the flexibility of the selection mechanism but also enhances its potential to adapt to a wider variety of problem instances. Also, unlike traditional hyper-heuristics that focus on selecting priority rules or constructing schedules, the proposed model selects among diverse metaheuristic search strategies based on project topology.

A. THE INPUTS OF THE DEVELOPED NEURAL NETWORK: PROJECT PARAMETRIC CHARACTERIZATIONS

The inputs of the developed neural network consist of three project network indices, namely NC, RF, and RS, which indicate the topological structure of the project. Initially, for each project, the number of activities (N) is defined as the total number of activities of the project excluding the dummy activities "Start" and "Finish" of the project.

The network complexity (NC) in project management refers to the number of activities and the total number of different combinations among them when creating a schedule within a project network. Specifically, it describes the relationships between various tasks that must be executed to achieve the project goals, which can be highly complex [44].

The resource factor (RF) refers to the average percentage of resources consumed across each resource category in the project [45].

$$RF = \frac{1}{J-2} \sum_{J=2}^{J-1} \frac{1}{Mj} \frac{1}{\tau} \sum_{m=1}^{Mj} \left| Qjm^{\tau} \right| \tag{5}$$

Finally, the term resource strength (RS) was introduced by Cooper [46] and measures resource availability K_r^T as a combination of the minimum K_r^{min} and maximum K_r^{max} levels of resource availability throughout the project duration [46], [47].

$$K_r^T = K_r^{min} + round(RS_T \left(K_r^{max} - K_r^{min} \right)) \tag{6}$$

The above project characteristics were selected for the development of the neural network because they describe and explain the constraints related to the average number of successors per unscheduled activity, the average resource requirements per activity, and the measurement of resource availability throughout the project execution [47].

B. THE PROPOSED APPROACH

The outputs of the neural network correspond to four distinct algorithms selected to represent diverse search strategies for solving the RCPSP: a Genetic Algorithm (GA), a Simulated Annealing algorithm (SA), a Bee Colony Optimization algorithm (BCO), and a Priority Rule-Based Heuristic (PRH). Instead of designing novel metaheuristics, this study employs well-established standard versions of these algorithms to focus on the efficacy of the selection model.

While implementing the metaheuristic algorithms, we used the random key representation, which is based on the concept of prioritizing activities. Random key representation employs a real random number $r \in [0, 1]$, which serves as the priority value for every activity of the project. To generate solutions for the metaheuristic algorithms, we employed the serial scheduling generation scheme (SSGS) proposed by Kelley [48]. This method was selected because of its superiority over the parallel method for creating schedules [3], [48].

The operational procedures of the algorithms are described below:

Simulated Annealing (SA) [49] represents a trajectory-based metaheuristic that avoids local optima through a

probabilistic cooling mechanism. The Genetic Algorithm (GA) [50] is an evolutionary, population-based method utilizing selection, crossover, and mutation operators. Bee Colony Optimization [51] relies on swarm intelligence principles, employing scout and recruit bees to balance exploration and exploitation. Finally, the priority rule based heuristic (PRH) is a constructive heuristic that employs the Serial Schedule Generation Scheme (SSGS) combined with standard priority rules (EST, LST, EFT, LFT) or random selection to generate schedules efficiently.

To ensure a fair comparison across these diverse methods, all algorithms were initialized to generate an equivalent total number of schedules (set to 5,000 schedules per instance). For the single-solution approaches (SA and PRH), this corresponded directly to the number of iterations. For the population-based methods (GA and BCO), the population size and number of generations were adjusted to match this total evaluation budget. This standardization ensures that any performance differences are attributable to the search strategy rather than computational resource disparities.

Finally, in this study, we employed metaheuristic algorithms based on different solution methods to provide insights not only into the most effective algorithm, but also into the most appropriate search strategy. For instance, the Genetic Algorithm is a population-based method that leverages evolutionary concepts. Similarly, the Bees Algorithm is also population-based but draws on principles of swarm intelligence. In contrast, Simulated Annealing is a trajectory-based algorithm. Lastly, the priority rule based heuristic algorithm does not belong to the metaheuristic category but is classified as a heuristic [52]. This diversity of algorithms ensures that instead of developing entirely new methods to solve the RCPSP, the model can recommend the most appropriate existing algorithm for a given project instance. Consequently, the user is empowered either to directly apply the suggested algorithm or to explore alternative algorithms that operate on the same underlying search mechanism (e.g., evolutionary, swarm intelligence, etc.).

C. THE DEVELOPED NEURAL NETWORK: CLASSIFICATION NEURAL NETWORK

To develop the neural network, the MATLAB software used, and specifically version R2023b. This version includes the ''fitcnet'' model, available since R2021a, which was employed to create the Classification Neural Network. This network is a Feedforward, Fully Connected Neural Network (FNN) trained for classification tasks. The first fully connected layer of the network receives the project characteristics NC, RF, and RS as input. Each subsequent layer is connected to the previous one. Every fully connected layer multiplies its input by a weight matrix (LayerWeights) and then adds a bias vector (LayerBiases). An activation function follows each fully connected layer (Activations and OutputLayerActivation). The final fully connected layer, together with the ''softmax'' activation function, produces

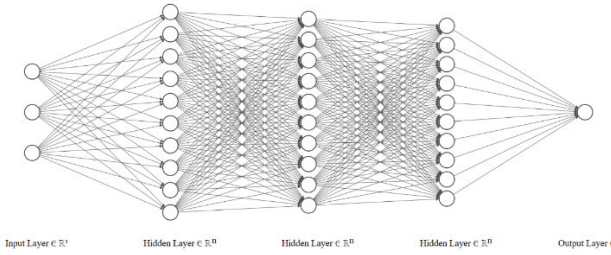


FIGURE 1. Feed-forward neural network architecture.

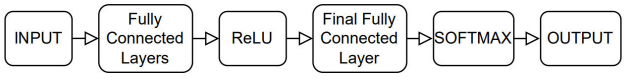


FIGURE 2. Neural network structure.

the network's output, which includes classification scores and predicted labels (yfit, scores) (Figure 6). In this study, these predicted labels correspond to the metaheuristic algorithms, thereby enabling the network to recommend the most suitable algorithm based on the input project characteristics.

The ReLU (Rectified Linear Unit) activation function is employed for the fully connected layers in the development of the neural network. This function performs a threshold operation on each input element, setting all negative values to zero. Mathematically, the ReLU function [53] is defined as:

$$f(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases} \tag{7}$$

Or

$$ReLU(x) = max(0, x) \tag{8}$$

The activation function for the final fully connected layer, returned as 'softmax', takes each input and returns the following:

$$f(x_i) = \frac{exp(x_i)}{\sum_{j=1}^{K} exp(x_j)} \tag{9}$$

where x is the input data and K is the number of classes [54].

The solver employed for training the neural network is referred to as LBFGS. In the development of the Classification Neural Network, a Limited-memory Broyden-Fletcher-Goldfarb-Shanno (LBFGS) quasi-Newton algorithm is utilized as the optimization technique for minimizing the loss function (Figure 7).

IV. APPLICATION TO THE PSPLIB SET OF PROBLEMS

The algorithms generating neural network training data include Simulated Annealing (SA), Bee Colony Optimization (BCO), Genetic Algorithm (GA), and Priority Rule-Based Heuristic (PRH). These solve RCPSP instances in PSPLIB and provide data for the FNN. We select PSPLIB projects with 60 and 120 activities.

PSPLIB is a library of project data generated through the ProGen generator. The projects have been created based on specific characteristics and are grouped accordingly. The characteristics of the projects used for training the neural network consist of the number of activities (N), network complexity (NC), resource factor (RF), and resource strength (RS) [47].

The dataset was constructed through the following process: Initially, project data from PSPLIB were grouped based on the number of activities. Within each activity group, projects are further organized into subsets of ten projects based on common values of Network Complexity (NC), Resource Factor (RF), and Resource Strength (RS). Each algorithm is provided with project subsets that share the same characteristics. The objective is for each algorithm to independently identify the optimal solution. This process enables a comparative evaluation of each algorithm's performance across projects with identical network characteristics. Analyzing the algorithms' results allows for evaluation of potential correlations between specific algorithms and the quality of the solutions obtained.

If all algorithms identified the same solution in a project subset, further comparison could use secondary criteria, such as shorter processing time or computational efficiency. To ensure mutually exclusive classes for the classification task and avoid label ambiguity during training, a strict hierarchy was applied. The ALL class (Class 5) was assigned if all algorithms score identical makespans. In such cases, the specific algorithm labels were suppressed, and the instance was strictly categorized as 'ALL'. This distinction is critical as it trains the network to recognize project topological features that render the specific choice of metaheuristic negligible, distinct from cases where a specific algorithm provides a measurable advantage.

A critical preprocessing step involved the resolution of label conflicts to ensure dataset consistency. Since the input features (NC, RF, RS) serve as high-level descriptors of project topology, it is possible for distinct project instances to share identical feature vectors while initially having different optimal algorithms due to latent structural differences. Such instances create 'one-to-many' mappings (conflicting labels) that introduce noise and hinder neural network convergence.

To eliminate ambiguity and ensure a deterministic mapping, the dataset underwent a rigorous refinement process. For project instances where multiple algorithms achieved the same optimal objective value, a tie-breaking rule was applied. Specifically, the instance was labeled based on the algorithm that demonstrated the superior aggregate performance across the entire set of ten projects for that specific configuration. This approach resolved potential label noise by assigning a single, consistent target to each feature vector, allowing the PCFNN to learn a well-defined classification boundary. The robust validation accuracy achieved further confirms that the model has captured meaningful patterns rather than overfitting, a result validated by the 10-fold cross-validation scheme.

TABLE 1. Algorithms coding.

Algorithm	Quantity
GA	1
BCO	2
PRH	3
SA	4
ALL	5

The developed neural network performs classification across five categories: SA, BCO, GA,PRH, and ALL. The 'ALL' category refers to the case described previously, in which all algorithms achieved the optimal solution. The encoding scheme used in the training files for the neural network is shown in Table 1:

A total of two neural networks were developed. The first network takes as input Network Complexity (NC), Resource Factor (RF), and Resource Strength (RS) for the J60 project instances from PSPLIB and produces as output the recommended algorithm. The second network is constructed identically, using NC, RF, and RS as inputs but applied to the J120 project instances from PSPLIB.

A. HYPERPARAMETER DERIVATION AND MODEL SELECTION

The architectural characteristics and hyperparameters of the proposed neural network were not manually tuned but were systematically derived through the MATLAB Classification Learner framework. To ensure optimal performance and generalization, we utilized the Optimizable Neural Network module, which automates the selection of model parameters.

To ensure the robustness of the proposed model and the reliability of the reported metrics, a 10-fold cross-validation scheme was implemented. This methodology partitions the dataset into ten disjoint subsets, the model is iteratively trained on nine subsets and validated on the tenth. This process is repeated ten times, ensuring that every data point is used for both training and validation. This rigorous approach confirms that the high accuracy achieved is not a result of overfitting and that the model generalizes effectively across the entire dataset. To eliminate any potential data leakage, the 10-fold cross-validation was performed at the configuration level rather than at the individual instance level. This ensures that instances sharing identical features were consistently kept together in either the training or validation folds.

The hyperparameter tuning process was driven by a Bayesian Optimization algorithm. This approach treats selection as a derivative-free optimization problem, iteratively exploring the parameter space to minimize validation error. Specifically, the model evaluated various configurations involving the number of fully connected layers, hidden layer sizes, activation functions, and the regularization strength λ to prevent overfitting.

By employing this automated approach within MATLAB, we ensured that the resulting model configurations represent

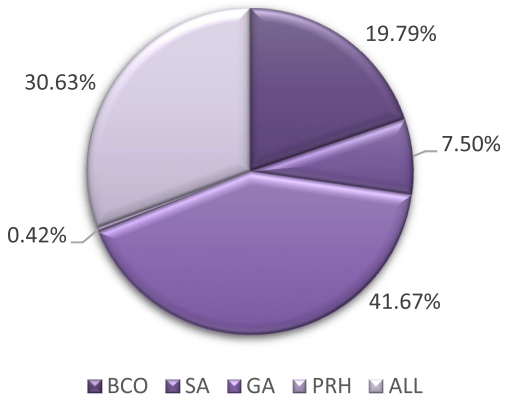


FIGURE 3. Algorithm comparison for J60 projects.

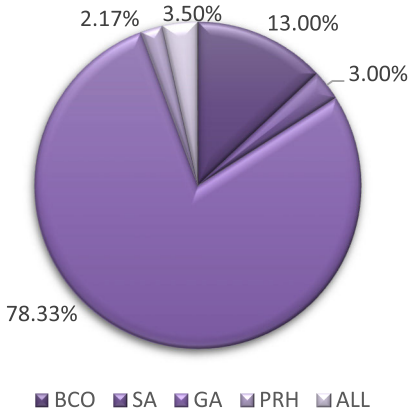


FIGURE 4. Algorithm comparison for J120 projects.

the most efficient architecture for the specific classification task. This methodology eliminates manual bias and provides a reproducible pathway for achieving the reported high accuracy.

B. THE PROPOSED APPROACH FOR TREATING J60, J120, J60&J120 NETWORKS

The performance comparison of the algorithms, including the integrated model, used to generate the training dataset for the J60 project instances from PSPLIB is presented in (Figure 3). Overall, the GA produced the best solutions, followed by the BCO, while the integrated model demonstrated its role as a complementary approach by providing additional comparative insight. Notably, across J60 instances, all algorithms achieved the optimal solution in 31% of cases. For the J120 project instances, the Genetic Algorithm again demonstrated superior performance, producing the best results in 78% of cases and significantly outperforms the other algorithms, as shown in (Figure 4).

The proposed Classification FNN (PCFNN) serves as an integrated model to select the optimal algorithm, achieving a validation accuracy of 98.4% for the PSPLIB J60 instances.

Although the initial search space allowed for up to three layers, the Bayesian optimization process determined that a more streamlined architecture yielded superior results. The final optimized model consists of a single fully connected layer with 300 neurons. The Rectified Linear Unit (ReLU) activation function was used, and the regularization strength λ was optimized to 1.446e-07. The final hyperparameter settings are detailed in Table 2.

TABLE 2. Hyperparameter settings – PCFNN/J60.

Parameter	Value
Model Type	Optimizable Neural Network (MATLAB)
Optimization Method	Bayesian Optimization
Number of Fully Connected Layers	1
Layer Size	300
Activation Function	ReLU
Iteration Limit	1000
Regularization (Lambda)	1.45E-07

TABLE 3. Performance metrics – PCFNN/J60.

Metric	Value
Accuracy (Validation)	98.40%
Error Rate	1.60%
Weighted Precision	98.40%
Weighted Recall	98.40%
Weighted F1-Score	98.30%
Prediction Speed	~20,000 obs/sec

The proposed model achieved an overall accuracy of 98.4% on the validation set. The training process was completed in 523.56 seconds, demonstrating a balance between high precision and computational efficiency. The prediction speed reached approximately 20,000 observations per second, making it suitable for real-time applications.

Detailed performance metrics, including Weighted F1-Score and Precision, are presented in Table 3.

The classification performance and per-class reliability are illustrated in the confusion matrix Figure 4 and the Positive Predictive Value (PPV) analysis Figure 5. The proposed model achieves an overall accuracy of 98.4%, demonstrating near-perfect identification for Classes 1, 2, 4, and 5. The observed zero metrics and the zero recall for Class 3 are attributed to the inherent performance disparity between the algorithms under study.

Specifically, Class 3-PRH represents a heuristic algorithm, which generally exhibits significantly lower performance and efficiency compared to the metaheuristic algorithms in the remaining classes. Due to this performance gap and the resulting scarcity of successful instances for this category, Class 3 was represented by only a single observation in the validation set, which was misclassified as Class 4. However, this localized discrepancy does not compromise the model's overall robustness. The high weighted F1-score (98.3%) and the 100% Precision (PPV) achieved for the primary metaheuristic

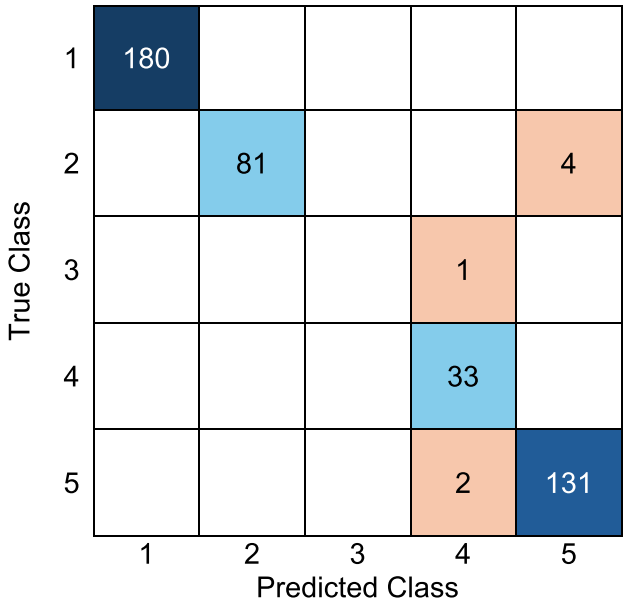


FIGURE 5. Confusion matrix of PCFNN/J60.

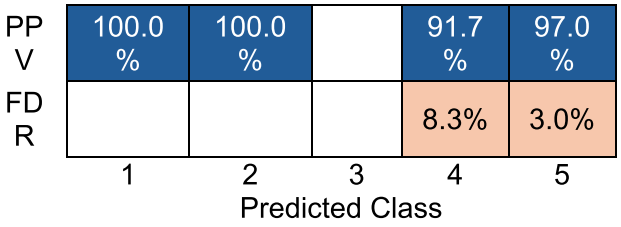


FIGURE 6. Positive predictive values (PPV) / False discovery rates (FDR) of PCFNN/J60.

classes confirm the network's capability to accurately distinguish between high-performing optimization approaches.

To assess the scalability and robustness of the Proposed Classification FNN (PCFNN), the experimental framework was extended to the J120 dataset. This dataset represents a more complex scheduling environment with 120 activities per project instance, significantly increasing the dimensionality of the classification task compared to the J60 baseline.

The Bayesian optimization process for the J120 instances resulted in a refined and computationally efficient architecture tailored to the problem's specific requirements. Interestingly, the optimized PCFNN/J120 model employs a more compact structure than its J60 counterpart, consisting of a single fully connected layer with 27 neurons. The ReLU activation function was selected to ensure effective non-linear mapping and stable gradient propagation. Furthermore, the regularization strength λ was precisely tuned to 2.354e-08, while input standardization was applied to maintain consistent feature scaling across the input space.

As shown in Table 5, the PCFNN maintained a high level of predictive power, achieving a validation accuracy of 92.4%. The weighted F1-score was recorded at 92.7%, with a weighted precision of 93.5%, demonstrating that the model

TABLE 4. Hyperparameter settings – PCFNN/J120.

Parameter	Value
Model Type	Optimizable Neural Network (MATLAB)
Optimization Method	Bayesian Optimization
Number of Fully Connected Layers	1
Layer Size	27
Activation Function	ReLU
Iteration Limit	1000
Regularization (Lambda)	2.354E-08

TABLE 5. Performance metrics – PCFNN/J120.

Metric	Value
Accuracy (Validation)	92.40%
Error Rate	7.60%
Weighted Precision	93.50%
Weighted Recall	92.40%
Weighted F1-Score	92.70%
Prediction Speed	~12,000 obs/sec

remains highly reliable even as the problem scale doubles. These metrics underscore the model's ability to generalize effectively to larger and more complex project instances.

To rigorously assess the utility of the proposed model, given the prevalence of the Genetic Algorithm (GA) in the J120 dataset (78.3%), we compared the PCFNN against a Naive Baseline strategy that consistently selects the majority class. While such an approach would yield a baseline accuracy of 78.3%, it possesses zero discriminatory power for alternative algorithms. In contrast, the PCFNN achieves a validation accuracy of 92.4%, representing a significant performance gain of 14.1% over the baseline. Crucially, this improvement is driven by the model's ability to correctly identify the minority classes (BCO, SA, and PRH) where the GA fails to provide the optimal solution. This confirms that the neural network has captured complex topological patterns rather than merely mirroring the underlying class distribution.

The training process for the J120 dataset was completed in 353.21 seconds. Notably, the model maintained a high prediction throughput of approximately 12,000 observations per second. This level of computational efficiency confirms that the PCFNN is well-suited for large-scale, real-time industrial applications, where rapid and automated algorithm selection is critical for optimizing project scheduling.

The classification performance and per-class reliability for the J120 dataset are further elucidated through the confusion matrix in Figure 6 and the Positive Predictive Value (PPV) / False Discovery Rate (FDR) analysis in Figure 7. As illustrated in the confusion matrix, the PCFNN demonstrates high diagnostic power across all categories, with the vast majority of observations correctly aligned along the main diagonal. Specifically, Class 1 (GA) exhibits remarkable stability, with

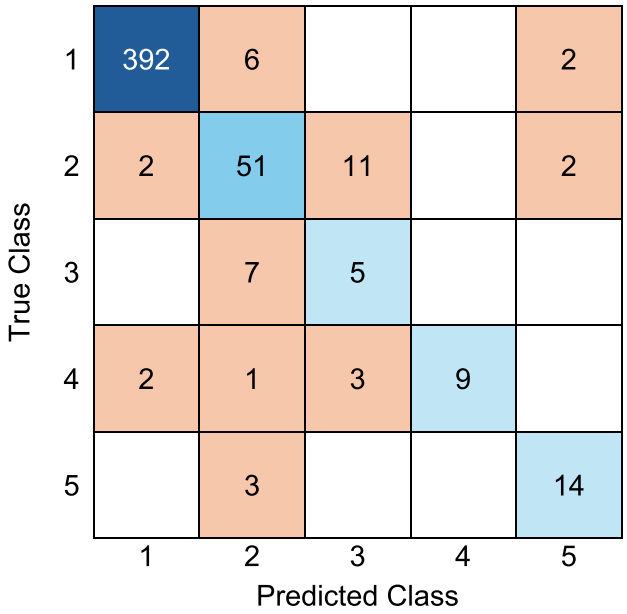


FIGURE 7. Confusion matrix of PCFNN/J120.

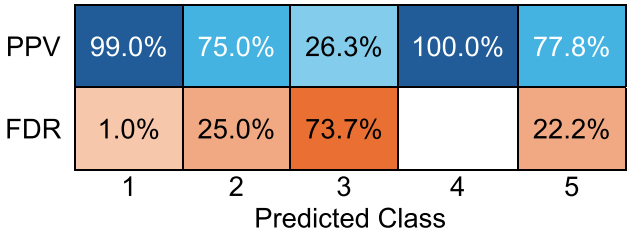


FIGURE 8. Positive predictive values (PPV) / False discovery rates (FDR) of PCFNN/J120.

392 instances correctly identified, resulting in a 99.0% PPV and a marginal 1.0% FDR.

A standout feature of the model's reliability is observed in Class 4, which achieves a perfect PPV of 100.0%, indicating that every recommendation for this class was invariably correct. Furthermore, the analysis reveals a significant breakthrough regarding Class 3 (PRH); while this category remained unidentifiable in the J60 instances, the PCFNN successfully captured patterns in the J120 search space, achieving 5 correct classifications. Although the 73.7% FDR for Class 3 reflects the inherent difficulty in distinguishing traditional heuristics from metaheuristics, its transition from an unclassifiable state to a quantifiable performance metric highlights the model's enhanced feature extraction capabilities at larger scales. Overall, these granular metrics confirm that the 92.4% validation accuracy is underpinned by strong per-class precision, particularly for the dominant metaheuristic algorithms.

V. APPLICATION IN A REAL CONSTRUCTION PROJECT

In addition to the PSPLIB library, the classification FNN was also applied to a real-world project involving the renovation

ID	Task Name	Duration in days	Predecessors	ID	TASK NAME	Duration in days	Predecessors
1	Resort Pools Construction			31	Engine Room Waterproofing Walls	2	30
2	Preparations-Site Organizing			32	Walls Road Metal Surroundings Settings	2	31
3	Contracts-Project Insurance	4		33	Eng. Room Ceiling Moulds	6	30
4	Installation in the Site Workplace	4		34	Stairways and Halls Concretes	3	29
5	Pools Complex Construction			35	Electromechanical Inft. For lights and Irrifation	3	34
6	Ground Water Drainage Study	2	4	36	Groundfloor Surroundings Concretes	4	34;35
7	Ground Water Drainage	30	6	37	Shafts Installations	3	34
8	Licenses from Archeological Office	10		38	Tanks-Walls Waterproofing	6	36
9	Concrete Elements Precipitations	4	8	39	Insulation Recovery-Trials	4	38
10	Excavation of Swimming Pool 1	6	8	40	Inside Waterproofing	10	18;19
11	Reclamation Layers Constructions	5	10	41	Swimming Pool 1 Edges Installations	6	40
12	Clearence Concretes	1	11	42	Stone Blocks Installations	7	41
13	Swimming Pool 1 Plate Ironing	2	12	43	Terrazzo Construction	7	42
14	Outside Wall Wooden Moulds	6	13	44	Swimming Pools Surface Paintings	6	43
15	Swimming Pool 1 Pipeworks	6	14	45	Stairways Roughcasts	4	47
16	Ground Floor Construction	3	15	46	Showers Installations	2	43
17	Inside Wall Wooden Moulds	4	16	47	Rustproof Elements Installations	2	46
18	Walls Construction	2	17	48	Steel Grits Installations	2	46;47
19	Walls Waterbrush Construction	2	18	49	Built-in Pipeworks - Trials	33	39
20	Perimetric Walls Coverings	2	18;19	50	Eng. Room Electrical-Plumbing Works	20	39
21	Spillway and Tank Coverings	4	20	51	Eng. Room Engines Installations	20	50
22	Engines Room Excavation	2	7	52	Surrondings Pipeworks	15	34;35;36
23	Reclamation Layers Constructions	2	20;21;22	53	Recovery Works	2	52
24	Clearence Concretes C12/15	1	23	54	Showers Shafts and Drainage Networks Construction	6	50
25	Ground Floor Waterproofing	1	24	55	Boreholds-Matrix-Pimps System	12	54
26	Clearence Concretes C16/20	1	25	56	Swomming Pools Lights and Edges Installations	2	44
27	Outside Wall Wooden Moulds	3	26	57	Electromechanical Equipment Installations	3	43
28	Engine Room Excavation Ground Floor Construction	2	25	58	Ground and Wall Lights Installations	12	55
29	Engine Room Inside Walls Construction	4	28	59	Surroundings Lights (Pillar Lights)	12	58
30	Engine Room Inside Walls Concretes	2	29	60	Plant Soil Settings	7	51;53;54;55;56;57;58;59

FIGURE 9. Project characteristics.

TABLE 6. Algorithms results in construction project.

Algorithm	Result
GA	152
BCO	152
PRH	152
SA	152
ALL	152

of a five-star hotel in Greece, specifically focused on the construction of a swimming pool complex. The details of this project are described in [55]. The examined project, according to the Critical Path Method (CPM), had an initial duration of 152 days.

For the presented project, Network Complexity (NC) was 1.3729, Resource Factor (RF) was 0.3158, and Resource Strength (RS) was 0.9333. These figures were calculated using formulas from [47] and [56] and were incorporated into the project's Patterson file. The performance results of the four tested algorithms are shown in Table 6 below.

The four algorithms were each initialized to generate 5000 schedules, leading to the same project duration. Table 3 summarizes the neural network's predictions for the input parameters NC = 1.3729, RF = 0.3158, and RS = 0.9333. Here, the PCFNN was trained using the J60 activity set, as it closely matched the project's number of activities. Regarding these specific instances, the PCFNN predicted Class 5. This output indicates a scenario where all candidate algorithms converge to the same solution, thereby confirming their collective suitability for the task at hand.

In this specific case, a random selection strategy or a naive baseline would have technically yielded the same result (152 days), given that all algorithms converged to the optimal duration. However, the distinct value of the PCFNN lies in its predictive confidence. Unlike a blind random choice, the model's Class 5 prediction explicitly informed the decision-making process that the project topology was simple enough for any search strategy to succeed. This insight eliminates the uncertainty and the need for extensive trial-and-error with multiple metaheuristics, confirming that a complex selection process was unnecessary.

VI. CONCLUSION

In this paper, we present a classification feedforward neural network (PCFNN), an artificial intelligence model developed to address the Resource-Constrained Project Scheduling Problem (RCPSP). This model focuses on identifying and recommending the most suitable algorithm within a predefined set for solving RCPSP instances. The advantages of using this model include the fact that it is trained only once to adjust the neural network weights, after which it can recommend optimal algorithms from the set on which it was trained. The alternative approach involves applying multiple algorithms and comparing their results to select the best one. Therefore, this model saves time for the user and guides the RCPSP solution towards specific algorithms. Results show that for projects with 60 activities, the model predicted the optimal algorithm with 98.4% accuracy, and 92.4% accuracy for those with 120 activities. In both the 60- and 120-activity project sets, the algorithm that consistently produced the best solutions was the Genetic Algorithm. While this algorithmic dominance highlights the effectiveness of the Genetic Algorithm, it also introduces a natural class imbalance within the dataset, requiring careful data handling to ensure the model reliably identifies the minority classes. The superior performance of this algorithm compared to the metaheuristic algorithms for projects with 60 and 120 activities may be attributed to the vast solution space, where the Genetic Algorithm effectively targeted regions with better solutions compared to the others. This observation, however, warrants further investigation.

The developed PCFNN is designed to predict the optimal algorithm for RCPSP instances with project characteristics and activity numbers identical to those found in the PSPLIB library, as the neural network was trained specifically with this dataset. This poses a potential limitation regarding the model's broader generalization to completely unknown project characteristics. To further enhance their predictive capability and generalization, future research is needed that involves training them on project generators with a broader combination of project characteristics as well as varying numbers of activities. Despite this fact the PCFNN managed to accurately predict the correct algorithm for solving the case study, even though its characteristics were different from those used for its training. Although the model can recommend only four algorithms, we believe this approach encourages researchers to use its features and develop similar models for algorithm comparison and recommendations in RCPSP solutions.

REFERENCES

[1] J. Blazewicz, J. K. Lenstra, and A. H. G. R. Kan, "Scheduling subject to resource constraints: Classification and complexity," *Discrete Appl. Math.*, vol. 5, no. 1, pp. 11–24, Jan. 1983.

[2] S. Colak, A. Agarwal, and S. Erenguc, "Resource constrained project scheduling: A hybrid neural approach," in *Perspectives in Modern Project Scheduling*. Boston, MA, USA: Springer, 2006, pp. 297–318, doi: 10.1007/978-0-387-33768-5_12.

[3] G. Koulinas, L. Kotsikas, and K. Anagnostopoulos, "A particle swarm optimization based hyper-heuristic algorithm for the classic resource constrained project scheduling problem," *Inf. Sci.*, vol. 277, pp. 680–693, Sep. 2014, doi: 10.1016/j.ins.2014.02.155.

[4] R. Pellerin, N. Perrier, and F. Berthaut, "A survey of hybrid metaheuristics for the resource-constrained project scheduling problem," *Eur. J. Oper. Res.*, vol. 280, no. 2, pp. 395–416, Jan. 2020, doi: 10.1016/j.ejor.2019.01.063.

[5] A. A. B. Pritsker, L. J. Waiters, and P. M. Wolfe, "Multiproject scheduling with limited resources: A zero-one programming approach," *Manage. Sci.*, vol. 16, no. 1, pp. 93–108, Sep. 1969. [Online]. Available: http://www.jstor.org/stable/2628369

[6] O. H. Türkakın, D. Arditi, and E. Manisalı, "Comparison of heuristic priority rules in the solution of the resource-constrained project scheduling problem," *Sustainability*, vol. 13, no. 17, p. 9956, Sep. 2021, doi: 10.3390/su13179956.

[7] A. Alorf, "A survey of recently developed metaheuristics and their comparative analysis," *Eng. Appl. Artif. Intell.*, vol. 117, Jan. 2023, Art. no. 105622, doi: 10.1016/j.engappai.2022.105622.

[8] K. Anagnostopoulos and G. Koulinas, "Resource-constrained critical path scheduling by a GRASP-based hyperheuristic," *J. Comput. Civil Eng.*, vol. 26, no. 2, pp. 204–213, Mar. 2012.

[9] A. Daliri, M. Alimoradi, M. Zabihimayvan, and R. Sadeghi, "World hyper-heuristic: A novel reinforcement learning approach for dynamic exploration and exploitation," *Expert Syst. Appl.*, vol. 244, Jun. 2024, Art. no. 122931, doi: 10.1016/j.eswa.2023.122931.

[10] K. P. Anagnostopoulos and G. K. Koulinas, "A simulated annealing hyperheuristic for construction resource levelling," *Construction Manage. Econ.*, vol. 28, no. 2, pp. 163–175, Feb. 2010, doi: 10.1080/01446190903369907.

[11] K. A. Dowsland, E. Soubeiga, and E. Burke, "A simulated annealing based hyperheuristic for determining shipper sizes for storage and transportation," *Eur. J. Oper. Res.*, vol. 179, no. 3, pp. 759–774, Jun. 2007, doi: 10.1016/j.ejor.2005.03.058.

[12] G. K. Koulinas and K. P. Anagnostopoulos, "A new Tabu search-based hyper-heuristic algorithm for solving construction leveling problems with limited resource availabilities," *Autom. Construction*, vol. 31, pp. 169–175, May 2013, doi: 10.1016/j.autcon.2012.11.002.

[13] J. Lin, L. Zhu, and K. Gao, "A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem," *Expert Syst. Appl.*, vol. 140, Feb. 2020, Art. no. 112915, doi: 10.1016/j.eswa.2019.112915.

[14] H. Chen, G. Ding, S. Qin, and J. Zhang, "A hyper-heuristic based ensemble genetic programming approach for stochastic resource constrained project scheduling problem," *Expert Syst. Appl.*, vol. 167, Apr. 2021, Art. no. 114174, doi: 10.1016/j.eswa.2020.114174.

[15] H. Chen, G. Ding, J. Zhang, R. Li, L. Jiang, and S. Qin, "A filtering genetic programming framework for stochastic resource constrained multi-project scheduling problem under new project insertions," *Expert Syst. Appl.*, vol. 198, Jul. 2022, Art. no. 116911, doi: 10.1016/j.eswa.2022.116911.

[16] K. P. Anagnostopoulos and G. K. Koulinas, "A genetic hyperheuristic algorithm for the resource constrained project scheduling problem," in *Proc. IEEE Congr. Evol. Comput.*, Jul. 2010, pp. 1–6, doi: 10.1109/CEC.2010.5586488.

[17] S. Hartmann, "A competitive genetic algorithm for resource-constrained project scheduling," *Nav. Res. Logistics (NRL)*, vol. 45, no. 7, pp. 733–750, 1998.

[18] S. Hartmann and R. Kolisch, "Experimental evaluation of state-of-the-art heuristics for the resource-constrained project scheduling problem," *Eur. J. Oper. Res.*, vol. 127, no. 2, pp. 394–407, Dec. 2000, doi: 10.1016/s0377-2217(99)00485-3.

[19] D. Bredael and M. Vanhoucke, "A genetic algorithm with resource buffers for the resource-constrained multi-project scheduling problem," *Eur. J. Oper. Res.*, vol. 315, no. 1, pp. 19–34, May 2024, doi: 10.1016/j.ejor.2023.11.009.

[20] T. Servranckx, J. Coelho, and M. Vanhoucke, "A genetic algorithm for the resource-constrained project scheduling problem with alternative subgraphs using a Boolean satisfiability solver," *Eur. J. Oper. Res.*, vol. 316, pp. 815–827, Mar. 2024, doi: 10.1016/j.ejor.2024.02.041.

[21] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[22] F. F. Boctor, "Resource-constrained project scheduling by simulated annealing," *Int. J. Prod. Res.*, vol. 34, no. 8, pp. 2335–2351, 1996.

[23] T. Bhattacharyya, B. Chatterjee, P. K. Singh, J. H. Yoon, Z. W. Geem, and R. Sarkar, "Mayfly in harmony: A new hybrid meta-heuristic feature selection algorithm," *IEEE Access*, vol. 8, pp. 195929–195945, 2020, doi: 10.1109/ACCESS.2020.3031718.

[24] D. T. Pham, A. Ghanbarzadeh, E. Koc, S. Otri, S. Rahim, and M. Zaidi, "The bees algorithm," in *Proc. ICINCO*, 2005, pp. 44–48.

[25] Y. Shi, F.-Z. Qu, W. Chen, and B. Li, "An artificial bee colony with random key for resource-constrained project scheduling," in *Proc. Int. Conf. Intell. Comput. Sustain. Energy Environ.*, 2010, pp. 148–157.

[26] R. Akbari, V. Zeighami, and K. Ziarati, "Artificial bee colony for resource constrained project scheduling problem," *Int. J. Ind. Eng. Computations*, vol. 2, no. 1, pp. 45–60, Jan. 2011, doi: 10.5267/j.ijiec.2010.04.004.

[27] R. Akbari, A. Mohammadi, and K. Ziarati, "A novel bee swarm optimization algorithm for numerical function optimization," *Commun. Nonlinear Sci. Numer. Simul.*, vol. 15, no. 10, pp. 3142–3155, Oct. 2010, doi: 10.1016/j.cnsns.2009.11.003.

[28] E. G. Talbi, *Metaheuristics: From Design To Implementation*. Hoboken, NJ, USA: Wiley, 2009, doi: 10.1002/9780470496916.

[29] F. Glover and G. A. Kochenberger, *Handbook of Metaheuristics Second Edition*. Boston, MA, USA: Kluwer Academic Publishers, 2003.

[30] Ö. Özkan and Ü. Gülçiçek, "A neural network for resource constrained project scheduling programming," *J. Civil Eng. Manage.*, vol. 21, no. 2, pp. 193–200, Jan. 2015, doi: 10.3846/13923730.2013.802723.

[31] P. Pagani and F. Pfann, "Deep neural networks for the scheduling of resource-constrained activity sequences: A preliminary investigation," *Logistics J.*, vol. 2020, 2020, doi: 10.2195/lj_Proc_pagani_en_202012_01.

[32] X. Zhao, W. Song, Q. Li, H. Shi, Z. Kang, and C. Zhang, "A deep reinforcement learning approach for resource-constrained project scheduling," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2022, pp. 1226–1234, doi: 10.1109/SSCI51031.2022.10022122.

[33] A. Golab, E. S. Gooya, A. A. Falou, and M. Cabon, "A multilayer feed-forward neural network (MLFNN) for the resource-constrained project scheduling problem (RCPSP)," *Decis. Sci. Lett.*, vol. 11, no. 4, pp. 407–418, 2022, doi: 10.5267/j.dsl.2022.7.004.

[34] A. Golab, E. S. Gooya, A. A. Falou, and M. Cabon, "A convolutional neural network for the resource-constrained project scheduling problem (RCPSP): A new approach," *Decis. Sci. Lett.*, vol. 12, no. 2, pp. 225–238, 2023, doi: 10.5267/j.dsl.2023.2.002.

[35] H. Cai, Y. Bian, and L. Liu, "Deep reinforcement learning for solving resource constrained project scheduling problems with resource disruptions," *Robot. Comput.-Integr. Manuf.*, vol. 85, Feb. 2024, Art. no. 102628, doi: 10.1016/j.rcim.2023.102628.

[36] P. Melchiors, R. Kolisch, and J. J. Kanet, "The performance of priority rules for the dynamic stochastic resource-constrained multi-project scheduling problem: An experimental investigation," *Ann. Oper. Res.*, vol. 338, no. 1, pp. 569–595, Feb. 2024, doi: 10.1007/s10479-024-05841-9.

[37] J. Luo, M. Vanhoucke, J. Coelho, and W. Guo, "An efficient genetic programming approach to design priority rules for resource-constrained project scheduling problem," *Expert Syst. Appl.*, vol. 198, Jul. 2022, Art. no. 116753, doi: 10.1016/j.eswa.2022.116753.

[38] W. Liu, J. Zhang, C. Qu, and H. Zhang, "Performance of priority rules for finance-based and resource-constrained project scheduling heuristics," *J. Construct. Eng. Manage.*, vol. 150, no. 6, Jun. 2024, doi: 10.1061/JCEMD4.COENG-14633.

[39] Z. Chen, E. Demeulemeester, S. Bai, and Y. Guo, "Efficient priority rules for the stochastic resource-constrained project scheduling problem," *Eur. J. Oper. Res.*, vol. 270, no. 3, pp. 957–967, Nov. 2018, doi: 10.1016/j.ejor.2018.04.025.

[40] J. H. Drake, A. Kheiri, E. Özcan, and E. K. Burke, "Recent advances in selection hyper-heuristics," *Eur. J. Oper. Res.*, vol. 285, no. 2, pp. 405–428, Sep. 2020, doi: 10.1016/j.ejor.2019.07.073.

[41] J. R. Rice, "The algorithm selection problem," *Adv. Comput.*, pp. 65–118, 1976, doi: 10.1016/s0065-2458(08)60520-3.

[42] T. Messelis and P. De Causmaecker, "An automatic algorithm selection approach for the multi-mode resource-constrained project scheduling problem," *Eur. J. Oper. Res.*, vol. 233, no. 3, pp. 511–528, Mar. 2014, doi: 10.1016/j.ejor.2013.08.021.

[43] E. L. Demeulemeester and W. S. Herroelen, *Project Scheduling: A Research Handbook*, vol. 49. Cham, Switzerland: Springer, 2006.

[44] N. Slack, S. Chambers, C. Harland, A. Harrison, and R. Johnston, *Operations Management*. New York, NY, USA: Pitman, 1998.

[45] T. L. Pascoe, "Allocation of resources CPM," *Revue Francaise de Recherche Operationnele*, vol. 10, no. 38, p. 31, 1966.

[46] D. F. Cooper, "Heuristics for scheduling resource-constrained projects: An experimental investigation," *Manage. Sci.*, vol. 22, no. 11, pp. 1186–1194, Jul. 1976, doi: 10.1287/mnsc.22.11.1186.

[47] R. Kolisch and A. Sprecher, "PSPLIB–A project scheduling problem library," *Eur. J. Oper. Res.*, vol. 96, no. 1, pp. 205–216, Jan. 1997.

[48] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," presented at the *Proc. IRE-AIEE-ACM*, vol. 1959, 1959, pp. 160–173.

[49] D. Bertsimas and J. Tsitsiklis, "Simulated annealing," *Stat. Sci.*, vol. 8, no. 1, pp. 10–15, Feb. 1993, doi: 10.1214/ss/1177011077.

[50] V. V. Peteghem and M. Vanhoucke, "A genetic algorithm for the preemptive and non-preemptive multi-mode resource-constrained project scheduling problem," *Eur. J. Oper. Res.*, vol. 201, no. 2, pp. 409–418, Mar. 2010.

[51] K. Ziarati, R. Akbari, and V. Zeighami, "On the performance of bee algorithms for resource-constrained project scheduling problem," *Appl. Soft Comput.*, vol. 11, no. 4, pp. 3720–3733, Jun. 2011.

[52] A. E. Ezugwu, A. K. Shukla, R. Nath, A. A. Akinyelu, J. O. Agushaka, H. Chiroma, and P. K. Muhuri, "Metaheuristics: A comprehensive overview and classification along with bibliometric analysis," *Artif. Intell. Rev.*, vol. 54, no. 6, pp. 4237–4316, Aug. 2021, doi: 10.1007/s10462-020-09952-0.

[53] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. ICML*, 2010, pp. 807–814.

[54] J. S. Bridle, "Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 1990, pp. 211–217.

[55] G. Koulinas, A. Xanthopoulos, T. Tsilipiras, and D. Koulouriotis, "Schedule delay risk analysis in construction projects with a simulation-based expert system," *Buildings*, vol. 10, no. 8, p. 134, Jul. 2020, doi: 10.3390/buildings10080134.

[56] E. Demeulemeester, M. Vanhoucke, and W. Herroelen, "RanGen: A random network generator for activity-on-the-node networks," *J. Scheduling*, vol. 6, no. 1, pp. 17–38, Jan. 2003, doi: 10.1023/a:1022283403119.



NIKOLAOS T. MITSOU received the Diploma (Dip.Eng.) degree in production and management engineering and the M.Sc. degree in innovation, technology and business management from the Democritus University of Thrace (D.U.Th.), Xanthi, Greece, in 2020 and 2023, respectively, where he is currently pursuing the Ph.D. degree in project management, focusing on the development of advanced artificial intelligence algorithms for solving project management problems. His research interests include artificial intelligence, machine learning, metaheuristic optimization, and their applications to project scheduling and intelligent decision-support systems.



GEORGIOS K. KOULINAS received the Diploma (Dip.Eng.) degree in production and management engineering from the Democritus University of Thrace, Greece, in 2006, the Ph.D. degree in production and management engineering from the Democritus University of Thrace in 2014, and the M.Sc. degree in construction project management from Hellenic Open University, Greece, in 2017. He is currently an Assistant Professor with the Department of Production and Management Engineering, Democritus University of Thrace, and an Adjunct Lecturer with Hellenic Open University. He has authored several scientific articles. His research interests include hyper-heuristic algorithms, metaheuristics, fuzzy systems, artificial intelligence, machine learning, risk management, project scheduling, project management, and safety systems.

The Open Access publication of this article was financially supported by HEAL-Link